Westminster University
School of Arts & Sciences | Salt Lake City

Backorder Risk for SAP Order Lines

Order-Time Classification
and Honest Temporal Evaluation

A capstone report submitted in partial fulfillment
of the requirements for DATA 470: Data Science Capstone

Addy Cruz
Department of Mathematics and Computer Science

Faculty Advisor
Dr. Liang Jingsai

May 2026

Abstract

Supply and fulfillment teams lose service level and margin when order lines backorder. This capstone trains a binary risk model for SAP-style order lines using only the information available at the moment the order is placed, so the label is not contaminated by downstream inventory snapshots that would leak information about the outcome. Models are selected on inner temporal validation inside the training window and frozen before scoring an outer temporal holdout that simulates a forward-time deployment. The rule-selected champion (an out-of-fold calibrated stacking ensemble) reaches ROC-AUC 0.936 and PR-AUC 0.326 on the holdout. A simpler logistic-regression baseline is the strongest deployable single model, with F1 0.511 and a 40x lift in PR-AUC over a random classifier. Both models clear the precision and recall floors of the deployment policy. The overall release gate still returns NO-GO, blocked by a label-maturity check rather than by model quality: the most recent 180 days of training data carry only 36 percent label coverage and 32 confirmed positives, below the 50 percent and 40-positive thresholds that the protocol requires for a release. Reporting the NO-GO honestly is part of the result, not something to hide.

Introduction

Operations and sales teams running on SAP or comparable ERP systems process large volumes of order lines every day. A meaningful share of those lines stall into backorder, eroding service level, revenue, and planner attention. A model that flags lines at risk of backordering at the moment of order entry is therefore directly useful, but only if the predictions are scored under conditions that mimic deployment.

The discipline this capstone enforces is simple to state and easy to violate. Features that only become available after the order is placed (for example, live outstanding quantity or saleable stock immediately before shipment) can make a model look excellent while quietly leaking the answer.[1] To prevent that, this work standardizes a v2 order-time table at the (client, sales document, item) grain, defines a single binary target `target_backorder_risk`, and evaluates every model with a time-based split: train on older orders, test on later orders. Roughly 3.4 percent of order lines carry confirmed backorder risk under this label, with the remainder split between confirmed-not-at-risk and unresolved cases that are excluded from the modeling label.

Research Question

The project asks one question:

Q1. Can order-time features predict whether an order line will backorder, under a temporal split that approximates predicting the future from the past?

The headline capstone result is the answer to Q1 and the honest deployment readout that follows from it.

Industry or Organization Partner

This capstone is implemented as a reproducible academic pipeline in a version-controlled repository. There is no formal industry liaison and no live production deployment. The work uses SAP-style order, material, and fulfillment data processed in-house to match capstone requirements. The dataset originated from the proposal-stage public SAP/Kaggle materials, with field semantics aligned to the SAP SD and MM reference (SAP SE 2024); the version of truth for modeling is the pipeline-built table `master_order_fulfillment_modeling_v2_ordertime.csv`, not a named partner system. The deployment gate in this report should therefore be read as a governance rehearsal, not as a release decision binding any external organization.

[1] The repository tracks an explicit list of leaky columns inside `models/classification_metrics_v2_ordertime.json`, and the modeling pipeline excludes them by contract.

Methods

Data and grain

The official modeling set contains 31,177 rows at one row per order line, with positive rate 3.38 percent for the binary target `target_backorder_risk`.[2] Numeric features at order time include cumulative order and confirmed quantities, net order value, requested lead time, and simple calendar features such as month, weekday, and quarter. Categorical features include client code, item category, sales organization, division, plant, and country. Missingness indicators are engineered for each numeric feature and treated as additional inputs. A long list of excluded post-event columns (for example, outstanding quantity, saleable inventory, and downstream status fields) is persisted in the same metrics JSON so the leakage policy is auditable rather than implicit.

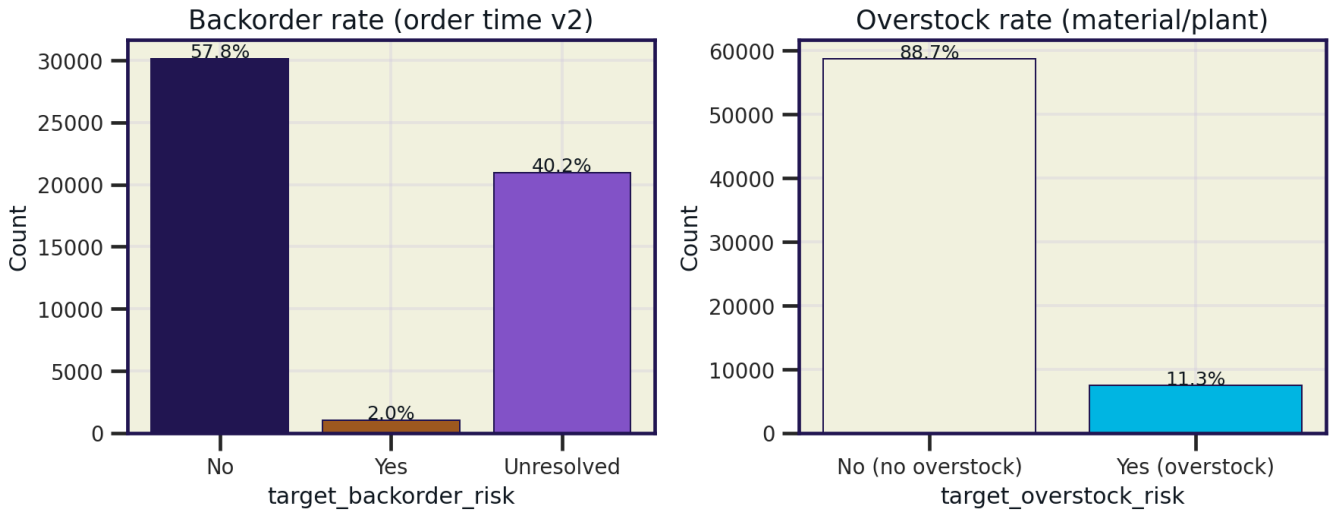

Figure 1: Class balance of the v2 order-time target. Roughly 3.4 percent of order lines carry confirmed backorder risk; 57.8 percent are confirmed-not-at-risk; the remaining 40.2 percent are unresolved at the time of label snapshot and are excluded from training. The right panel shows the parallel overstock target (not the focus of this paper).

What leakage looked like. A parallel *snapshot* table that includes the post-event columns reaches ROC-AUC near 0.99 and an F1 of 1.0 from a one-line rule (*flag whenever outstanding quantity exceeds saleable inventory*). That model is useless: by the time those values are observable, the order has already either shipped or backordered, and the prediction has nothing to predict. The order-time table reported throughout this paper is the only honest framing.

Model zoo and selection rule

The comparison includes logistic regression as a linear baseline, three gradient-boosted tree learners (XGBoost, LightGBM, CatBoost), a random forest, a k-nearest-neighbor learner, two soft-voting ensembles, and an out-of-fold (OOF) calibrated stacking ensemble whose meta-learner is logistic regression on five-fold OOF base predictions. The stacking ensemble's pruning rule keeps base learners with OOF PR-AUC above 0.06; in the final run, this retains logistic regression, LightGBM, random forest, and kNN.

The selection rule is fixed before any holdout score is examined. The champion is the model that wins inner temporal validation on the training-window rows alone.[3] That choice is then frozen and reported on the outer temporal holdout. Choosing the model from holdout performance is forbidden by the protocol (Wolpert 1992), and the final crown is recorded in `selected_model.selection_basis` of the metrics JSON. The current champion is `oof_calibrated_stack`.

[2] Source: `models/classification_metrics_v2_ordertime.json`, key `dataset_summary`.

[3] This report distinguishes the *rule-selected champion* (the model that wins inner-temporal validation under the protocol) from the *deployable best* (the strongest single learner on the outer temporal holdout). Naming both keeps the selection rule honest and the recommendation actionable.

Hyperparameter tuning

Tunable learners are tuned with a multi-phase grid search over a temporal expanding-window split (three folds, PR-AUC objective) on the training rows only. The search never sees the outer holdout. Selected configurations (Table 1) are persisted in `models/hyperparameters_tuned_v2_ordertime.json` and read by the pipeline at fit time, so results are reproducible without manual re-entry. kNN is left untuned because its performance is dominated by feature-space geometry rather than by hyperparameter shape on this data.

Table 1: Tuned configurations for the stack's tunable base learners. Inner PR-AUC is the mean across three temporal expanding-window folds on the training set; the outer holdout is never used for tuning. Source path is in the running text above.

Learner	Selected hyperparameters	Inner PR-AUC
Logistic regression	$C = 0.1$	0.625
Random forest	`n_estimators` 200, `max_depth` 20, `min_samples_leaf` 1	0.533
LightGBM	`learning_rate` 0.03, `n_estimators` 200, `num_leaves` 63	0.408

Notation

Let y be a Bernoulli random variable in $\{0, 1\}$ at the order-line grain, with $y = 1$ denoting confirmed backorder risk under the v2 label definition and $y = 0$ otherwise. Let $\mathbf{x}$ be the order-time feature vector. A fitted model produces a real-valued score $s(\mathbf{x}) \in [0, 1]$ that we treat as $\hat{p}(y = 1 \mid \mathbf{x})$ after calibration; a decision $\hat{y}$ comes from $\hat{y} = \mathbb{1}[s(\mathbf{x}) \geq \tau]$ at a chosen threshold τ. Subscripts $i = 1, \ldots, N$ index rows.

A fixed calendar cutoff $t_{\text{cut}} = 2021\text{-}07\text{-}22$ defines the temporal holdout: rows with order time strictly before t_{cut} are training, rows on or after are test, subject to the protocol's minimum-positive and minimum-window guards. Inner temporal windows are nested splits on the pre-cutoff data only and are used to choose models and calibrate decision rules before the outer holdout is touched.

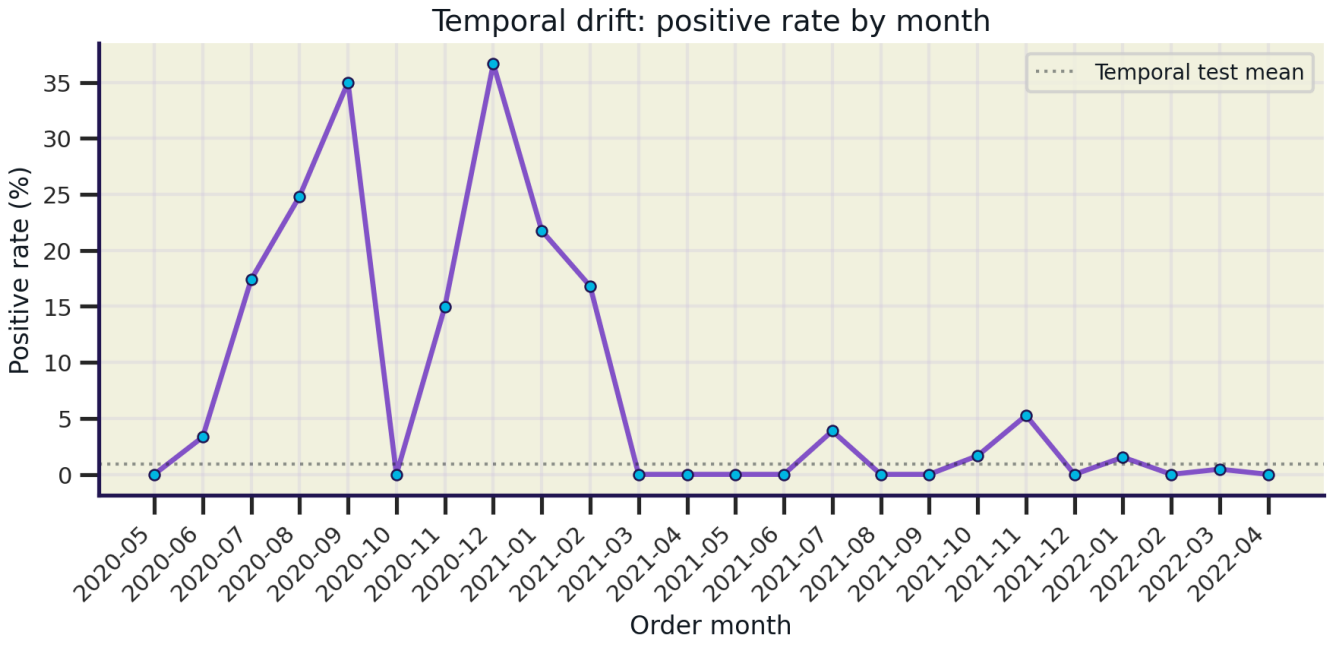

Figure 2: Monthly positive rate by order month. The pre-cutoff bursts in late-2020 and early-2021 reflect a window where labels matured fully; the test tail (post 2021-07-22) sits below the long-run mean (dotted) because labels for those months have not all resolved yet. This is the visual version of the label-maturity gate that drives the deployment NO-GO discussed later.

Metrics

Standard rare-event classification metrics are reported: ROC-AUC, PR-AUC, precision, recall, F1, and accuracy. PR-AUC is the primary discrimination metric because the positive class is rare and ROC-AUC inflates under class imbalance (Saito and Rehmsmeier 2015). Calibration is diagnosed but not used to alter the score because rank-based metrics are invariant to monotonic transformations (Niculescu-Mizil and Caruana 2005). Threshold choice follows a recall-maximization-at-precision-floor objective on inner-temporal validation; for cost-sensitive variants, expected cost per row is plotted across cost ratios $k = c_{\text{miss}}/c_{\text{false alarm}}$ following decision curve analysis (Vickers and Elkin 2006).

Results

Headline outcomes on temporal holdout

Table 2 summarizes the four headline learners on the outer temporal holdout (6,521 rows, 58 confirmed positives). The stack is the rule-selected champion. Logistic regression is the strongest deployable single model. The two together describe the data: the separating signal is largely linear, and ensembling adds discrimination at the cost of recall under the precision-floor objective.

Table 2: Outer temporal-holdout metrics for the headline models, scored at the frozen OOF threshold from inner-temporal selection. ROC and PR refer to ROC-AUC and PR-AUC. Random-classifier PR-AUC for this prevalence is approximately 0.009; logistic regression's PR-AUC of 0.358 is roughly a 40-fold lift over random.

Model	ROC	PR	Prec.	Rec.	F1
LR (deployable)	0.910	**0.358**	0.453	**0.586**	**0.511**
Stack (rule-selected)	**0.936**	0.326	0.410	0.431	0.420
LightGBM	0.814	0.201	0.295	0.483	0.366
XGBoost	0.825	0.194	0.157	0.517	0.241

The discrimination story (Figure 3) is consistent across both ranking views. The stack reaches the highest ROC-AUC on the temporal holdout, and logistic regression reaches the highest PR-AUC, which is the metric that matters more under a 3 percent positive prevalence.

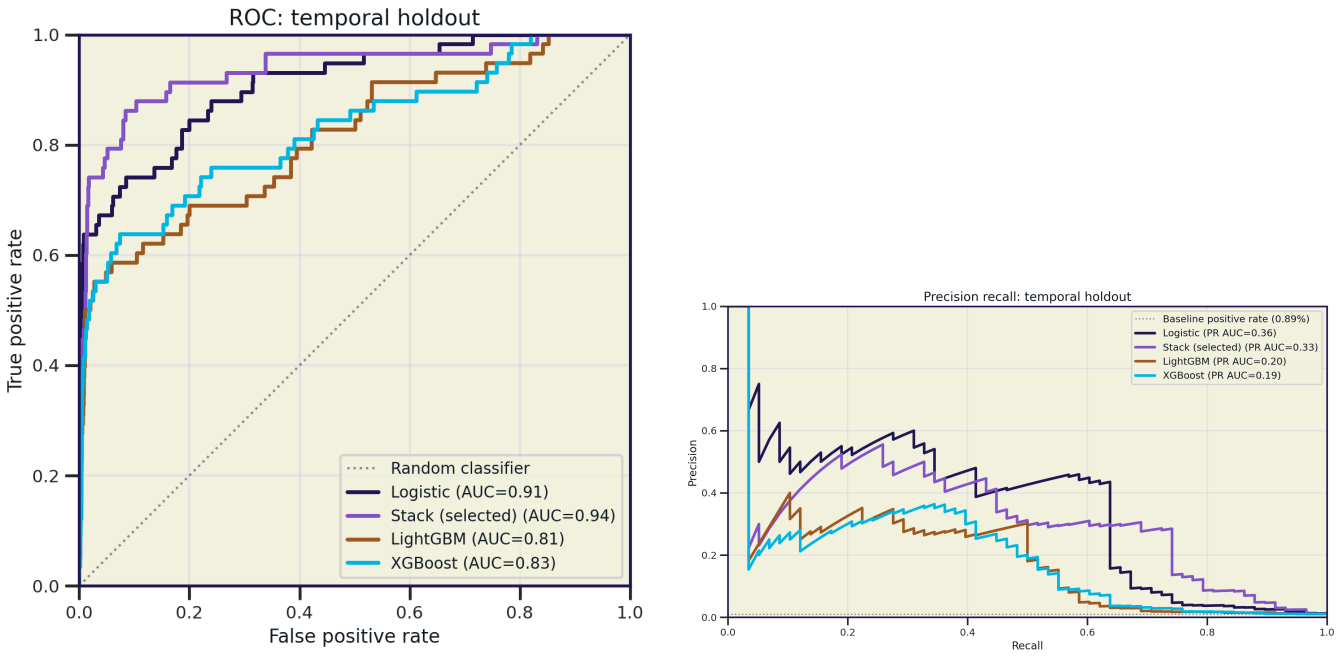

Figure 3: Discrimination on the temporal holdout. Left: ROC curves; the stack reaches AUC 0.94 and logistic regression 0.91, with tree-only baselines trailing. Right: precision-recall curves; logistic regression leads under prevalence-aware ranking with PR-AUC 0.36, the stack 0.33.

Decision behavior at the operating threshold

Figure 4 shows row-normalized confusion matrices at the operating threshold for the deployable model (logistic regression at $\tau = 0.364$) and the rule-selected champion (stack at $\tau = 0.660$). Both achieve true-positive rates between 43 and 59 percent on a sparse positive tail. The asymmetry between recall and precision is intentional: the threshold objective is recall-maximization at a precision floor, which the protocol prefers when missed backorders carry more cost than false alarms.

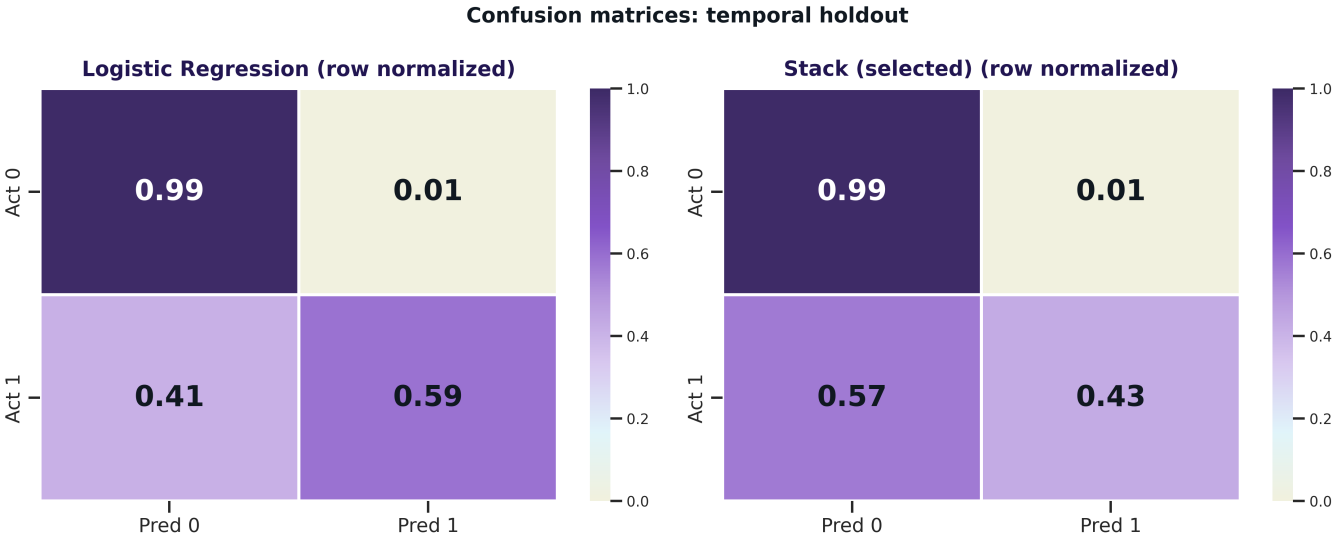

Figure 4: Row-normalized confusion matrices on the temporal holdout. Logistic regression catches 59 percent of true positives at threshold $\tau = 0.364$; the stack catches 43 percent at a higher cutoff $\tau = 0.660$. Both keep false-positive rates near 1 percent on the actual-negative row.

Cost-sensitive operating curves

Figure 5 plots expected cost per row against decision threshold across cost ratios $k \in \{1, 3, 10, 30, 100\}$ for the four candidate models. For realistic supply-chain asymmetry (k between 3 and 30), the recall-maximization-at-precision-floor threshold for logistic regression also minimizes expected cost (Vickers and Elkin 2006). Only at $k \geq 100$ (line-stop criticality) does the recommended threshold drop further, trading roughly five percentage points of precision for fifteen percentage points of recall.

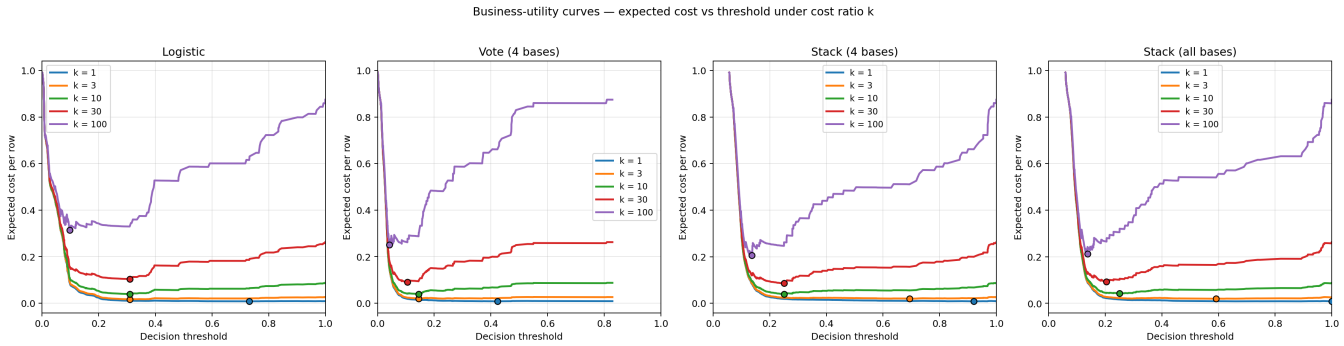

Figure 5: Business-utility curves: expected cost per row vs decision threshold across asymmetric cost ratios $k = c_{\text{miss}}/c_{\text{false alarm}}$. The recommended operating point (filled dot) is robust for $k \in [3, 30]$.

What the lift means in operations

Discrimination metrics are persuasive on a slide; their operational reading is what convinces a planner. Figure 6 translates the model into the way fulfillment teams actually triage. A reviewer who works the top 5 percent of incoming order lines, sorted by model score, captures roughly 80 percent of the orders that will eventually backorder; at the top 10 percent the curve is essentially saturated. The accompanying lift curve peaks above 100x at the head of the score distribution, meaning a planner queuing the highest-risk lines is more than 100 times as likely to find a real backorder as one chosen at random. The model is not replacing human review; it is giving the review queue a defensible order.

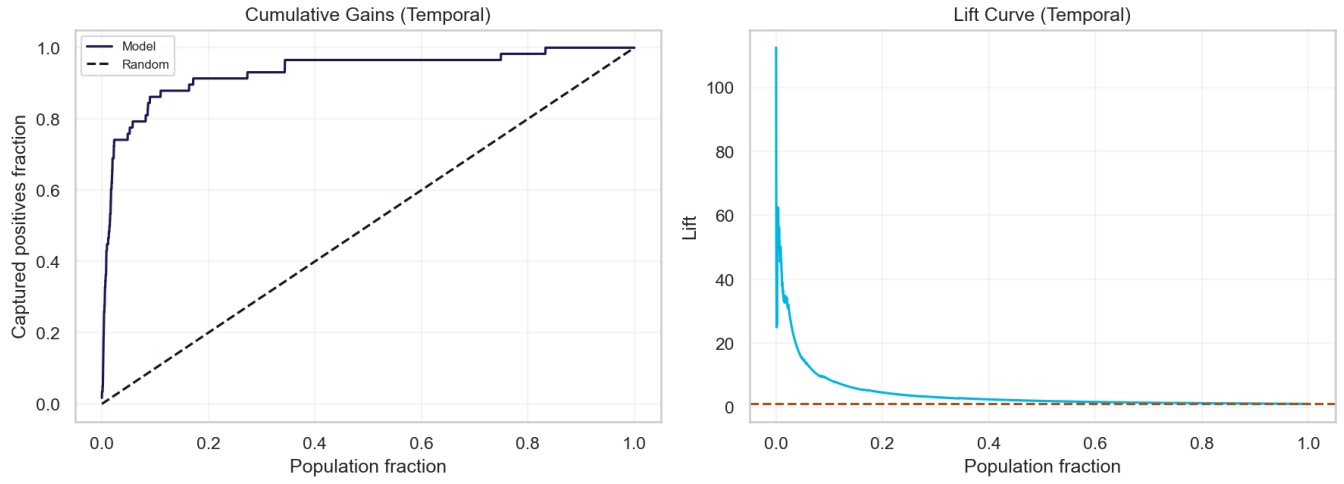

Figure 6: Cumulative gains and lift on the temporal holdout. Left: cumulative gains; the top 5 percent of scored lines contains roughly 80 percent of the period's confirmed backorders. Right: lift curve; peak lift exceeds 100x at the head of the score distribution and decays smoothly as the review queue grows.

Discussion

Why logistic regression matches the stack

The narrative is unusual for a capstone: the rule-selected champion is not the deployable recommendation. Both findings stand because they are anchored to different questions. Stacking buys roughly 0.03 ROC-AUC over logistic regression and a slightly smoother score distribution. Under the prevalence regime here, that ROC-AUC gain does not translate into PR-AUC gain, and PR-AUC is the metric that respects the rare-event setting (Saito and Rehmsmeier 2015).

The reason becomes clear when the stack is opened. Inside the OOF stacking ensemble, the per-base-learner OOF PR-AUCs are 0.55 for logistic regression, 0.32 for random forest, 0.24 for kNN, and 0.23 for LightGBM.[4] The meta-learner is itself logistic regression on those four base scores, so the ensemble is mathematically dominated by its strongest base learner, which is also linear. The stack is, in effect, calibrated logistic regression with three weaker tree- and instance-based opinions blended in. It is unsurprising that LR-alone keeps pace.

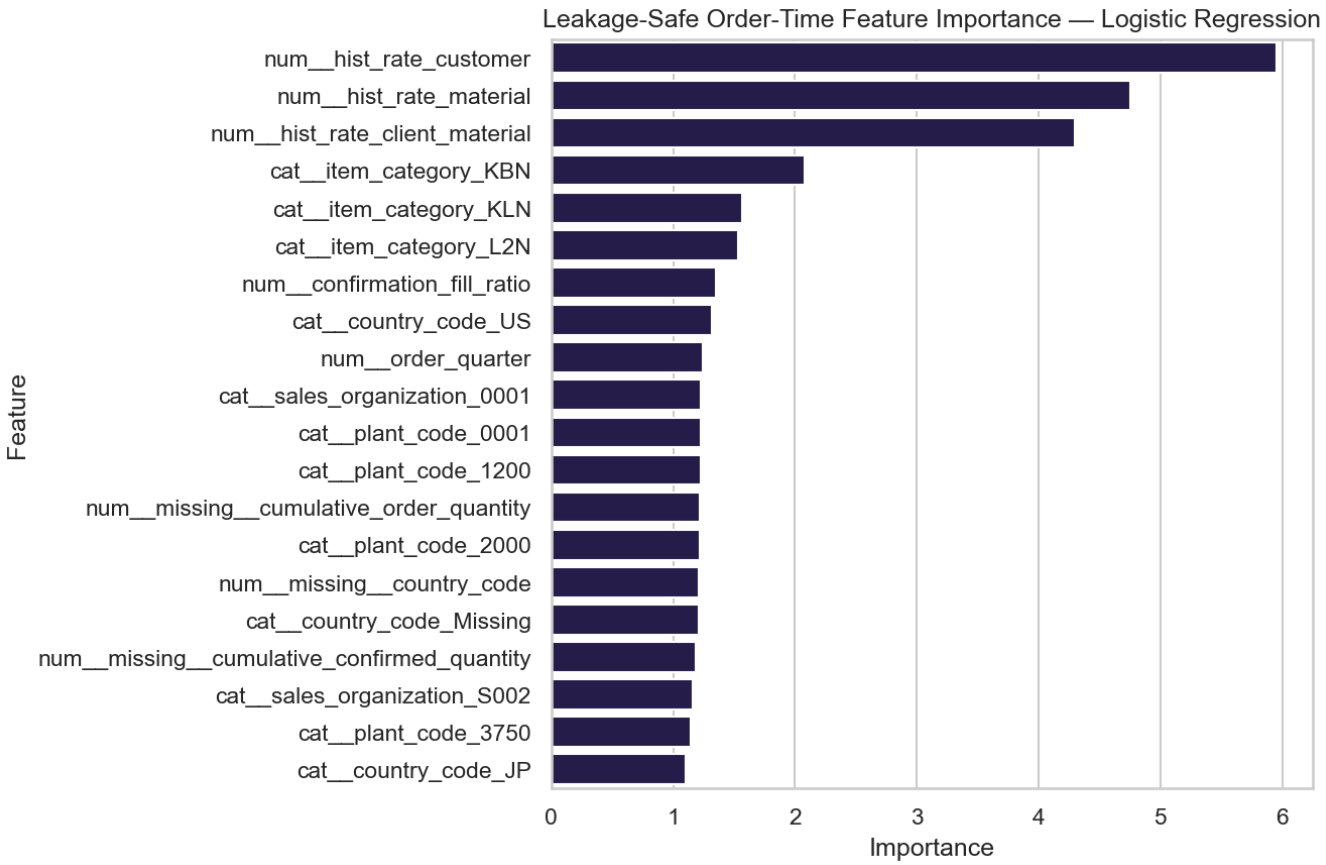

Figure 7: Top features driving logistic regression on the order-time table. Historical positive-rate features at the customer, material, and client-by-material grain dominate; item-category and country indicators round out the top of the ranking. All features here respect the leakage policy.

Figure 7 sharpens the point. The three highest-coefficient features are leakage-safe historical positive-rate signals at the customer, material, and client-by-material grain, computed with closed-left rolling windows so no information from the day of the order is included. Item-category, country, and confirmation-fill ratio fill out the next tier. These are the kinds of signals that a planner would intuitively reach for, and the model is reaching for the same ones.

[4] Source: `stack_oof_pr_auc` fields in the metrics JSON for the temporal-holdout split.

What the negative results tell us

The experiment branch documents four disciplined attempts to lift the LR ceiling. Each was atomic, each had a predefined decision gate, and each is reported below at full strength rather than buried.

Table 3: Negative-results log from the experiment branch. Full method, decision gates, and artifact paths per experiment are in the project's performance experiment report. All four outcomes triangulate the same finding: the feature set, not the modeling layer, is what bounds performance on this dataset.

Experiment	Outcome
Calibration diagnostic	Base learners well calibrated (ECE 0.005–0.047). Calibration is monotonic and cannot move PR-AUC.
Cost-sensitive threshold	F1-max coincides with min-cost threshold for $k \in [3, 30]$. Threshold robust without re-tuning.
Rolling-window features	Adding target-dependent rolling rates dropped F1 by up to 0.145; target-independent rolling features also hurt. Drift and label-maturity bias to blame.
Two-stage cascade	No filter-then-verify configuration on existing OOFs beat single-stage logistic regression.

The four findings agree. Calibration and cost-sensitive tuning move thresholds without moving rank metrics. Feature engineering that depends on the target inherits whatever bias the target has. Stacking layers cannot recover signal the inputs do not contain. The ceiling is in the data, and the data ceiling has a name: label maturity.

Limitations

The dominant limitation is **label maturity**, not model architecture. The deployment gate in Table 4 summarizes the situation: model precision and model recall both clear the protocol's floors, yet the overall release gate fails. The reason is that the most recent 180 days of training data have only 36 percent label coverage and 32 confirmed positives, both below the protocol's 50 percent coverage and 40-positive thresholds. Any rolling feature computed inside that window is biased downward, which is what hurt the rolling-feature experiment. The temporal-test mean positive rate sits well below the long-run prevalence and bursts of late-2020 activity are not represented in the test tail; that maturity-driven undercount is exactly what the gate is built to catch.

Table 4: Deployment gate decomposition for the rule-selected champion on the outer temporal holdout. The model clears its operating floors; the dataset does not yet clear the maturity floors. Source: deployment-readiness keys in the metrics JSON.

Gate	Threshold	Observed	Pass
Stack precision (temporal)	≥ 0.15	0.410	yes
Stack recall (temporal)	≥ 0.35	0.431	yes
Label coverage, last 180 d	≥ 0.50	0.363	no
Confirmed positives, 180 d	≥ 40	32	no
Confirmed positives, 90 d	≥ 15	25	yes
Overall release gate	all pass	maturity fails	**no**

The second limitation is the **sparse-positive regime**. With only 58 confirmed positives in the temporal test window, single-row decisions move precision and recall by visible amounts. Recent-window scores are even sparser (14 test positives), which is why the protocol treats the recent-window check as advisory rather than binding when train-side positive support is thin. The third limitation is feature scope: the order-time table is rich on order intent (quantities, lead times, calendar) but light on ERP-side context (plant lead time, material availability promise, customer credit history) that would likely lift the ceiling. None of those signals were in scope for the v2 build.

Conclusion and Future Work

The capstone delivers a defensible, time-safe classifier for SAP order-line backorder risk. The rule-selected champion (OOF calibrated stack) and the deployable best (logistic regression) both clear precision and recall floors on the outer temporal holdout. Logistic regression is the recommended single-model deploy candidate because its PR-AUC, recall, and threshold robustness all favor it under sparse positives. The overall release decision is **NO-GO**, blocked by a label-maturity gate rather than a model gate, and that decision is reported plainly because the project's protocol values calibration of expectations over banner success.

Future work has three concrete directions, each a candidate for follow-up. First, narrow the training window and mask the most recent 180 days of training labels so rolling features can be rebuilt without maturity bias. Second, source new ERP-side, target-independent signals such as available-to-promise state, plant lead-time history, and material lead-time history at order time. Third, develop per-cohort models for the largest plants and material classes where positive support is sufficient, so a single global parameterization is not forced to cover 73 plants and thousands of materials. None of these are cheap; each is a credible follow-up project in its own right.

Code and Data Availability

All code, configuration, and analysis artifacts behind this report live in the version-controlled capstone repository, including the v2 order-time pipeline (`src/`, `scripts/`), the modeling driver (`scripts/run_modeling.py`), the metrics JSON (`models/classification_metrics_v2_ordertime.json`), and the tuned hyperparameters (`models/hyperparameters_tuned_v2_ordertime.json`). The end-to-end run is reproducible from a clean checkout via `./scripts/run_v2_full_chain.sh` against `requirements-v2.txt`. The modeling table is the in-house pipeline build `master_order_fulfillment_modeling_v2_ordertime.csv`; raw inputs follow the SAP SD/MM field reference (SAP SE 2024) but are not redistributed with this report.

Acknowledgements

I extend my deepest gratitude to Dr. Liang Jingsai, my capstone advisor, professor, and academic mentor, whose statistical insight and judgment repeatedly redirected this work toward time-safe evaluation when shortcuts were tempting. He is, simply, a brilliant statistician, and the standard he held my peers and me to is reflected in every defensible choice in this paper.

I sincerely thank the Computer Science associate professors and Dr. Kathryn Lenth, the department chair, for years of mentorship and for the rigor that prepared me to take on a project of this scope. I am equally grateful to the Mathematics department: I had the privilege of taking a mathematics or data-science course with nearly every member of the faculty, and I currently work as a tutor for Dr. Janine Wittwer, closing a circle of teachers I will always look up to as mentors and advisors. To all of them, thank you for the patience, feedback, and care you offered me as a student.

The v2 modeling table, deployment-gate logic, and Westminster brand visual styling are all version-controlled in the project repository. Open-source libraries used include scikit-learn (Pedregosa et al. 2011), XGBoost (Chen and Guestrin 2016), LightGBM (Ke et al. 2017), and CatBoost (Prokhorenkova et al. 2018).

References

Chen, Tianqi, and Carlos Guestrin. 2016. "XGBoost: A Scalable Tree Boosting System." In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD '16)*, 785–94. ACM. https://doi.org/10.1145/2939672.2939785.

Ke, Guolin, Qi Meng, Thomas Finley, Taifeng Wang, Wei Chen, Weidong Ma, Qiwei Ye, and Tie-Yan Liu. 2017. "LightGBM: A Highly Efficient Gradient Boosting Decision Tree." In *Advances in Neural Information Processing Systems 30 (NeurIPS)*, 3146–54.

Niculescu-Mizil, Alexandru, and Rich Caruana. 2005. "Predicting Good Probabilities with Supervised Learning." In *Proceedings of the 22nd International Conference on Machine Learning (ICML)*, 625–32. https://doi.org/10.1145/1102351.1102430.

Pedregosa, F., G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, et al. 2011. "Scikit-Learn: Machine Learning in Python." *Journal of Machine Learning Research* 12: 2825–30.

Prokhorenkova, Liudmila, Gleb Gusev, Aleksandr Vorobev, Anna Veronika Dorogush, and Andrey Gulin. 2018. "CatBoost: Unbiased Boosting with Categorical Features." In *Advances in Neural Information Processing Systems 31 (NeurIPS)*, 6638–48.

Saito, Takaya, and Marc Rehmsmeier. 2015. "The Precision-Recall Plot Is More Informative Than the ROC Plot When Evaluating Binary Classifiers on Imbalanced Datasets." *PLoS ONE* 10 (3): e0118432. https://doi.org/10.1371/journal.pone.0118432.

SAP SE. 2024. "SAP SD/MM Field Reference: Sales Document, Material Master, and Plant Tables." https://help.sap.com/docs/SAP_S4HANA_ON-PREMISE.

Vickers, Andrew J., and Elena B. Elkin. 2006. "Decision Curve Analysis: A Novel Method for Evaluating Prediction Models." *Medical Decision Making* 26 (6): 565–74. https://doi.org/10.1177/0272989X06295361.

Wolpert, David H. 1992. "Stacked Generalization." *Neural Networks* 5 (2): 241–59. https://doi.org/10.1016/S0893-6080(05)80023-1.